

Quiero que construyas un proyecto llamado `journal-review-simulator-web`.

Este proyecto debe ser una plataforma web local, desplegable con Docker, que simule un sistema editorial de revista/congreso científico similar a un portal de submission/review. El objetivo es ayudar a preevaluar papers antes de enviarlos a revistas o congresos reales.

El sistema debe permitir:

1. Crear una revisión con ID único.
2. Subir un paper.
3. Detectar si es una submission nueva o una resubmission.
4. Subir reviews anteriores y carta/respuesta a revisores.
5. Extraer el paper a Markdown.
6. Clasificar el área y tipo de paper.
7. Sugerir revistas/congresos candidatos.
8. Configurar revistas, congresos, papers recientes y literatura de estado del arte.
9. Generar prompts para motores externos como ChatGPT, Claude, Gemini, Perplexity y NotebookLM.
10. Permitir que el usuario suba respuestas externas en Markdown, PDF o DOCX.
11. Ejecutar un pipeline autónomo con subagentes/revisores independientes.
12. Permitir iteraciones/pedidos de información al usuario.
13. Generar decisión editorial simulada por revista.
14. Generar plan de revisión, estrategia de rebuttal y carta final.
15. Guardar historial completo de cada revisión.
16. Mantener todo el conocimiento principal en archivos `.md`, `.yaml`, `.csv` y `.json`, con indexación en base de datos.

El proyecto debe usar Claude Code como constructor y orquestador principal. El archivo principal de instrucciones debe llamarse `CLAUDE.md`.

Principio arquitectónico central

Usa esta regla:

Markdown como memoria documental. Base de datos como índice y estado. Web app como workflow. Agentes como evaluadores. Docker como despliegue reproducible. GitHub privado como repositorio de código y plantillas.

Los documentos `.md` son la fuente semántica principal para agentes, skills y revisión. La base de datos solo debe indexar, buscar, filtrar, relacionar y controlar estados.

Stack recomendado

Implementa una arquitectura modular con:

- Frontend web: Next.js o React.
- Backend API: FastAPI.
- Worker de pipeline: Python.

- Base de datos: PostgreSQL.
- Modelos locales opcionales: Ollama.
- Orquestación: Docker Compose.
- Documentos: Markdown, YAML, CSV, JSON.
- Exportación: Markdown, PDF y ZIP.
- Repositorio: preparado para GitHub privado.

Si prefieres una alternativa técnica por simplicidad, propónla, pero mantén estos requisitos funcionales.

Reglas críticas

- Nunca inventes referencias bibliográficas.
- Nunca inventes cuartiles, rankings, impact factor, CiteScore, SJR, h-index, acceptance rate, review time o publication time.
- Si un dato no está verificado, escribe `UNKNOWN` , `NOT_VERIFIED` , `MISSING` o `NEEDS_USER_INPUT` .
- Distingue siempre entre:
 - evidencia extraída del paper
 - inferencia del revisor
 - datos externos verificados
 - datos faltantes
 - recomendación editorial
- Si no hay literatura real aportada, la evaluación de novedad debe marcarse como provisional.
- Si no hay datos reales del venue, la evaluación del venue debe marcarse como provisional.
- No descargues papers de pago ni contenido con copyright sin autorización.
- No simules una revisión real por pares. Esto es una herramienta de ayuda pre-submission.
- Cada revisor debe trabajar de forma independiente.
- El editor debe sintetizar desacuerdos; no debe promediar mecánicamente.
- Todas las salidas deben guardarse en Markdown, YAML, CSV o JSON.
- Cada revisión debe tener un ID único.
- Cada archivo subido debe quedar vinculado al ID de revisión.
- Cada output debe indicar modelo, motor, reviewer, venue, fuentes usadas y limitaciones.
- El sistema debe ser modular, auditable y fácil de ampliar.
- La subida de archivos debe ser segura: validar extensión, tipo real, tamaño, nombre, rutas internas de ZIPs y posibles ficheros peligrosos.

Áreas de investigación del usuario

El sistema debe estar preparado para analizar papers y buscar venues relacionados con estas familias de investigación.

1. Cloud, distributed systems and computer architecture

- computer architecture
- high-performance computing
- distributed systems
- cloud computing
- edge computing
- serverless computing

- Kubernetes
- orchestration
- cloud task scheduling
- CloudSim
- CloudSim Plus
- resource allocation
- load balancing
- priority scheduling
- starvation-free scheduling
- weighted fair queuing
- datacenter simulation
- energy-aware computing
- green computing
- hardware security
- RISC-V
- heterogeneous architectures
- GPU / TPU / FPGA education
- digital sovereignty
- EU Chips Act
- computer architecture curricula

2. Machine learning, deep learning and robust generalization

- machine learning
- deep learning
- computer vision
- transfer learning
- domain adaptation
- domain generalization
- out-of-distribution generalization
- covariate shift
- concept shift
- calibration transfer
- grouped cross-validation
- leakage-free evaluation
- benchmark design
- data leakage
- robust evaluation protocols
- model-free diagnostics
- discriminant analysis
- geometric diagnostics
- representation learning
- interpretable machine learning
- explainable AI

3. Face anti-spoofing, biometrics and security

- face anti-spoofing
- presentation attack detection
- biometric recognition
- biometric security

- face recognition security
- video-based identity verification
- spoofing detection
- replay attack detection
- print attack detection
- deepfake detection
- 3D convolutional neural networks
- spatiotemporal neural networks
- video classification
- residual spatiotemporal networks
- C(2+1)D
- R3D
- M3D
- CSN
- feature map visualization
- explainable AI for video models
- interpretable deep learning
- lightweight FAS models
- real-time FAS
- computational cost vs accuracy trade-off

4. Handwritten Text Recognition, OCR and Document AI

- handwritten text recognition
- HTR
- OCR
- document analysis
- document image analysis
- document AI
- handwriting recognition
- offline handwriting recognition
- child handwriting recognition
- student handwriting recognition
- low-resource language HTR
- Catalan HTR
- multilingual HTR
- synthetic handwriting generation
- synthetic data for HTR
- handwriting datasets
- line-level HTR datasets
- writer-independent evaluation
- document understanding
- text line recognition
- transformer-based HTR
- curriculum learning for HTR
- fine-tuning for HTR
- data augmentation for handwriting
- benchmark datasets for HTR
- annotation protocols
- transcription protocols
- educational worksheets

- K-12 handwriting datasets

5. Educational technology, computing education and engineering education

- computing education
- computer science education
- programming education
- introductory programming
- CS1 education
- K-12 computing education
- STEM education
- engineering education
- educational technology
- learning technologies
- computational thinking
- algorithmic thinking
- natural language programming
- pseudocode-based programming
- block-based programming
- natural-language programming environments
- conversational programming
- AI-supported programming education
- paper-based programming
- programming learning barriers
- syntax barriers
- novice programming
- educational interventions
- systematic literature reviews in education
- PRISMA systematic reviews
- curriculum analysis
- CS2023
- ACM/IEEE curriculum guidelines
- computer architecture education
- RISC-V education
- hardware security education
- digital sovereignty education

6. Agricultural AI, hyperspectral imaging, food safety and chemometrics

- agricultural AI
- precision agriculture
- wheat detection
- wheat quality assessment
- food safety
- food quality
- mycotoxin detection
- deoxynivalenol detection
- DON screening

- Fusarium detection
- cereal contamination
- near-infrared hyperspectral imaging
- NIR-HSI
- hyperspectral imaging
- spectroscopy
- chemometrics
- single-kernel grain analysis
- non-destructive testing
- calibration transfer
- radiometric calibration
- spectral machine learning
- batch effect
- acquisition-session drift
- laboratory session generalization
- data descriptor
- scientific datasets
- benchmark datasets
- FAIR data
- reproducible agricultural benchmarks

7. Dataset, benchmark, software and data descriptor papers

El sistema debe reconocer papers cuyo principal aporte sea un dataset, benchmark, corpus, plataforma, software o recurso reproducible.

Debe buscar venues adecuados para:

- data descriptor papers
- dataset papers
- benchmark papers
- corpus papers
- software papers
- reproducibility papers
- resource papers
- open science papers
- FAIR data papers
- benchmark protocol papers
- evaluation protocol papers

El sistema debe evaluar si el paper encaja mejor como:

- research article
- data descriptor
- software article
- benchmark paper
- survey
- systematic review
- education research article
- applied AI article
- systems paper

- computer vision paper
- document analysis paper
- agricultural sensing paper
- food safety / chemometrics paper

Familias de venues que el sistema debe buscar

El sistema debe buscar y clasificar venues en estas familias:

1. Computer architecture / HPC / cloud / distributed systems.
2. Machine learning / AI / deep learning.
3. Computer vision / pattern recognition.
4. Biometrics / face anti-spoofing / presentation attack detection / security.
5. Document analysis / OCR / HTR / document AI.
6. Educational technology.
7. Computing education / CS education.
8. Engineering education.
9. Agricultural AI / precision agriculture.
10. Hyperspectral imaging / remote sensing.
11. Food safety / food quality / chemometrics.
12. Domain adaptation / OOD generalization / robust ML.
13. Dataset / benchmark / data descriptor venues.
14. Software / reproducibility / open science venues.

Estructura del proyecto

Crea esta estructura:

```
journal-review-simulator-web/ |— CLAUDE.md |— README.md |— docker-compose.yml
|— .env.example |— .gitignore |— apps/ | |— web/ | | |— README.md | | |—
package.json | | |— app/ | | |— components/ | | |— lib/ | | |— styles/ | | |—
api/ | | |— README.md | | |— pyproject.toml | | |— app/ | | |— main.py | | |— api/
| | |— models/ | | |— schemas/ | | |— services/ | | |— repositories/ | | |—
security/ | | |— tests/ | | |— worker/ | | |— README.md | | |— pipeline_runner.py | | |—
agent_orchestrator.py | | |— external_prompt_manager.py | | |— file_ingestion.py | | |—
review_id.py | | |— markdown_store.py | | |— exporters.py | | |— config/ | | |— pipeline.yaml
| | |— scoring_rubrics.yaml | | |— model_config.yaml | | |— reviewer_profiles.yaml | | |—
venue_discovery.yaml | | |— external_engines.yaml | | |— upload_policy.yaml | | |— db/ | | |—
migrations/ | | |— schema.sql | | |— seed/ | | |— data/ | | |— global_knowledge/ | | |—
venues/ | | | |— journals/ | | | |— conferences/ | | | |— template/ | | | |—
recent_papers/ | | | |— literature/ | | | |— reviewer_profiles/ | | | |— prompt_templates/ |
| | | |— reviews/ | | | |— README.md | | | |— uploads/ | | | |— quarantine/ | | | |— prompts/ |
| | | |— external_engines/ | | | |— chatgpt/ | | | |— claude/ | | | |— gemini/ | | | |—
perplexity/ | | | |— notebooklm/ | | | |— reviewer_prompts/ | | | |— venue_discovery/
| | | |— .claude/ | | | |— agents/ | | | |— editor-in-chief.md | | | |— venue-fit-analyst.md | |
| | | |— venue-discovery-analyst.md | | | |— venue-timeline-analyst.md | | | |— literature-
scout.md | | | |— reviewer-methodology.md | | | |— reviewer-domain.md | | | |— reviewer-
systems-architecture.md | | | |— reviewer-reproducibility.md | | | |— reviewer-scientific-
impact.md | | | |— reviewer-document-ai-htr.md | | | |— reviewer-education-research.md | | |
```

```

├── reviewer-dataset-benchmark.md | | ├── reviewer-spectral-agri-food.md | | ├──
reviewer-fas-biometrics-security.md | | ├── reviewer-cloud-hpc-scheduling.md | | ├── format-
compliance-checker.md | | ├── integrity-ai-use-auditor.md | ├── skills/ | ├── review-paper-
reviewer/ | ├── SKILL.md ├── scripts/ | ├── run_pipeline.py | ├── parse_paper.py | ├──
discover_venues.py | ├── scan_venue_markdowns.py | ├── build_literature_matrix.py | ├──
compare_models.py | ├── build_report.py | ├── export_review_package.py | ├──
init_review.py | ├── tests/

```

Organización interna por ID de revisión

Cada revisión debe tener un ID único. Usa un formato legible y único, por ejemplo:

REV-YYYYMMDD-HHMMSS-<short_random_id>

Ejemplo:

REV-20260607-143012-A8F3K2

Cada revisión debe crear una carpeta:

```

data/reviews/<review_id>/ ├── metadata.yaml ├── input/ | ├── original/ | ├──
previous_reviews/ | ├── author_response/ | ├── upload_report.md ├── extracted/ | ├──
manuscript_extracted.md | ├── paper_extraction.json | ├── latex_extraction_report.md | ├──
references_extracted.bib ├── venues/ | ├── selected_venues.yaml | ├── venue_fit_report.md
| ├── venue_timeline_report.md | ├── venue_snapshots/ ├── literature/ | ├──
literature_matrix.csv | ├── candidate_papers.bib | ├── notes/ ├── external_prompts/ | ├──
index.md | ├── by_venue/ ├── external_responses/ | ├── index.md | ├── by_venue/
| ├── pending_requests/ | ├── pending_requests.md | ├── pending_requests.json ├──
reviewer_outputs/ | ├── internal/ | ├── specialized/ | ├── external_summaries/ ├──
editor/ | ├── editor_decision.md | ├── meta_review.md | ├── revision_plan.md | ├──
rebuttal_strategy.md | ├── final_letter.md ├── exports/ | ├── final_review_package.zip |
| ├── editor_decision.pdf | ├── revision_plan.pdf ├── audit/ ├── audit_log.md ├──
file_hashes.json ├── model_usage.md ├── limitations.md

```

Base de datos

La base de datos debe indexar el contenido documental. No debe sustituir los `.md`.

Crea tablas o modelos equivalentes para:

- reviews
- papers
- uploaded_files
- extracted_documents
- venues
- venue_files
- literature_items
- recent_papers

- reviewer_profiles
- external_engines
- external_prompts
- external_responses
- internal_agent_runs
- pending_requests
- editorial_decisions
- audit_events

Campos mínimos para `reviews` :

- id
- review_id
- title
- status
- submission_type
- created_at
- updated_at
- selected_venue_id
- paper_type
- detected_areas
- current_step
- final_decision
- path_on_disk

Campos mínimos para `venues` :

- id
- venue_id
- name
- acronym
- type
- venue_family
- quartile_or_rank
- q1_accessibility_class
- publication_speed_category
- review_rigor
- official_url
- path_on_disk
- last_verified_at

Campos mínimos para `external_prompts` :

- id
- review_id
- venue_id
- reviewer_profile
- engine
- prompt_file_path
- expected_response_filename
- status
- created_at

Campos mínimos para `pending_requests` :

- id
- review_id
- venue_id
- agent_id
- request_type
- question
- generated_prompt_path
- status
- iteration_number
- max_iterations

Diseño web

Crea una interfaz clara, elegante y académica.

Estilo visual:

- colores claros
- fondo gris muy claro
- tarjetas blancas
- azul oscuro, verde petróleo o azul IEEE como color principal
- acentos sutiles
- tipografía limpia
- diseño sobrio, no recargado

Debe tener:

Menú lateral

- Dashboard
- Reviews
- New Review
- Papers
- Venues
- Recent Papers / SOTA
- External Prompts
- Pending Requests
- Reviewer Profiles
- AI Engines
- Settings
- Export / History

Menú superior dentro de cada revisión

- 1. Create / Metadata
- 1. Upload
- 1. Extraction
- 1. Area & Paper Type

- 1. Venues
- 1. Desk-Reject Precheck
- 1. Reviewer Profiles
- 1. External Prompts
- 1. External Responses
- 1. Autonomous Review
- 1. Integrity Audit
- 1. Editor Decision
- 1. Revision Plan
- 1. Export

Página de configuración de venues

Debe permitir:

1. Crear nueva revista/congreso.
2. Editar revista/congreso.
3. Importar desde CSV.
4. Rellenar formulario manual.
5. Generar automáticamente los archivos `.md`, `.yaml` y `.json` necesarios.
6. Subir aims and scope.
7. Subir author guidelines.
8. Subir editorial policy.
9. Subir review criteria.
10. Subir template/formato de la revista.
11. Subir papers recientes de la revista.
12. Subir ejemplos de artículos aceptados.
13. Generar prompts para ChatGPT, Claude, Gemini, Perplexity y NotebookLM para completar información faltante.
14. Mostrar campos no verificados.
15. Mostrar fecha de última verificación.
16. Mostrar quartile/rank y fuente.
17. Mostrar tiempo de publicación y fuente.
18. Mostrar riesgo de desk reject.
19. Mostrar clase de accesibilidad Q1:
 - elite Q1
 - strong Q1
 - mid-tier Q1
 - accessible Q1
 - borderline Q1/Q2
 - strong Q2
 - unknown

Estructura de un venue

Cada venue debe guardarse como documentos.

data/global_knowledge/venues/journals/<venue_id>/ |—— venue_profile.yaml |—— overview.md
 |—— aims_and_scope.md |—— author_guidelines.md |—— editorial_policy.md |——

review_criteria.md |—— indexing_and_quartile.md |—— publication_timeline.md |——
 q1_accessibility.md |—— format_requirements.md |—— recent_papers/ |—— example_papers/
 |—— verification_log.md

Para conferencias:

data/global_knowledge/venues/conferences/<venue_id>/ |—— venue_profile.yaml |—— overview.md
 |—— call_for_papers.md |—— review_criteria.md |—— ranking.md |—— important_dates.md
 |—— proceedings_info.md |—— format_requirements.md |—— recent_papers/ |——
 example_papers/ |—— verification_log.md

venue_profile.yaml

Usa esta estructura:

venue_id: UNKNOWN name: UNKNOWN acronym: UNKNOWN type: journal_or_conference
 publisher_or_organizer: UNKNOWN official_url: UNKNOWN field:

- UNKNOWN venue_family:
- UNKNOWN scope_summary: UNKNOWN article_types:
- UNKNOWN quartile: source: UNKNOWN year: UNKNOWN category: UNKNOWN quartile:
 UNKNOWN conference_rank: source: UNKNOWN year: UNKNOWN rank: UNKNOWN indexing:
 scopus: UNKNOWN web_of_science: UNKNOWN dblp: UNKNOWN core_rank: UNKNOWN
 google_scholar_metrics: UNKNOWN metrics: impact_factor: UNKNOWN cite_score: UNKNOWN
 sjr: UNKNOWN h_index: UNKNOWN q1_accessibility: class: UNKNOWN rationale: UNKNOWN
 confidence: UNKNOWN realistic_publication_chance: UNKNOWN
 recommended_submission_strategy: UNKNOWN acceptance_rate: UNKNOWN review_style:
 UNKNOWN expected_contribution_level: UNKNOWN methodological_rigor: UNKNOWN
 experimental_rigor: UNKNOWN reproducibility_expectation: UNKNOWN publication_timeline:
 time_to_first_decision: UNKNOWN time_to_acceptance: UNKNOWN time_acceptance_to_online:
 UNKNOWN estimated_total_publication_time: UNKNOWN publication_speed_category:
 UNKNOWN fast_track_available: UNKNOWN online_first_available: UNKNOWN
 expected_review_rounds: UNKNOWN timeline_confidence: UNKNOWN timeline_source:
 UNKNOWN review_difficulty: rigor_level: UNKNOWN desk_reject_risk: UNKNOWN selectivity:
 UNKNOWN common_delay_reasons:
 - UNKNOWN suitable_for_fast_publication: UNKNOWN common_rejection_reasons:
- UNKNOWN good_fit_for:
- UNKNOWN bad_fit_for:
- UNKNOWN paper_profiles_supported:
- research article
- systematic review

- dataset paper
- data descriptor
- software article
- benchmark paper
- education research article
- applied AI article
- systems paper
- document analysis paper
- agricultural sensing paper
- food safety / chemometrics paper

Página de configuración de papers recientes / SOTA

Debe permitir subir papers recientes por área.

El sistema debe:

1. Aceptar PDF, Markdown, DOCX, BibTeX o notas.
2. Extraer metadatos.
3. Clasificar el paper por área.
4. Relacionarlo con una o varias revistas.
5. Relacionarlo con uno o varios perfiles de reviewer.
6. Guardarlo como `.md`.
7. Indexarlo en base de datos.
8. Marcar si el paper es:
9. foundational
10. recent SOTA
11. dataset paper
12. benchmark paper
13. competing method
14. negative result
15. related venue example
16. challenge paper
17. survey / systematic review

Estructura:

```
data/global_knowledge/recent_papers/<paper_id>/ |—— paper_profile.yaml |——
extracted_summary.md |—— relevance_by_area.md |—— relevance_by_reviewer.md |——
relevance_by_venue.md |—— citation.bib |—— source_file_info.md
```

Página de AI engines

Debe permitir configurar:

- Claude Code
- Claude API, opcional
- Ollama
- ChatGPT manual
- Gemini manual
- Perplexity manual
- NotebookLM manual

Modos:

- manual_external_prompt
- autonomous_local
- autonomous_claude_code
- hybrid

Por defecto:

- Claude Code activado para pipeline interno.
- Ollama activado si está disponible.
- Motores web externos en modo manual.

Cada respuesta de reviewer debe indicar:

- engine
- model
- mode
- reviewer_profile
- venue_id
- review_id
- data_sources_used
- limitations
- confidence

Seguridad de uploads

Implementa una fase de validación segura antes de procesar archivos.

Debe:

1. Validar extensión permitida.
2. Validar tipo MIME real.
3. Limitar tamaño.
4. Renombrar archivo con nombre interno seguro.
5. Calcular hash SHA256.
6. Guardar archivo original en carpeta controlada.
7. Descomprimir ZIPs solo en sandbox.

8. Bloquear rutas peligrosas en ZIPs.
9. Bloquear archivos ejecutables o inesperados.
10. Detectar ficheros ocultos sospechosos.
11. Generar `upload_report.md`.
12. Registrar todo en `audit/file_hashes.json`.

Formatos permitidos:

- .pdf
- .docx
- .md
- .tex
- .zip
- .bib
- .csv
- .yaml
- .json

Formatos de entrada del paper

El sistema debe aceptar:

- `manuscript.md`
- `manuscript.pdf`
- `manuscript.docx`
- `manuscript.tex`
- `manuscript.zip`

La prioridad de extracción debe ser:

1. LaTeX
2. Markdown
3. DOCX
4. PDF

Si existe LaTeX y PDF, debe preferirse LaTeX porque conserva mejor estructura, secciones, referencias, ecuaciones y comandos.

Entrada LaTeX

Si el input es un archivo `.tex`, leer directamente el LaTeX y extraer el contenido científico principal.

Entrada ZIP con LaTeX

Si el input es un `.zip`, el sistema debe:

1. Descomprimirlo en:

data/reviews/<review_id>/input/latex_source/

1. Detectar automáticamente el archivo principal de LaTeX. Posibles nombres:

2. main.tex

3. manuscript.tex

4. paper.tex

5. article.tex

6. Si hay varios `.tex`, intentar identificar el principal buscando:

7. `\documentclass`

8. `\begin{document}`

9. `\title`

10. `\author`

11. `\abstract`

12. `\input{}`

13. `\include{}`

14. Resolver de forma básica archivos incluidos mediante:

15. `\input{...}`

16. `\include{...}`

17. Extraer o reconstruir las secciones principales:

18. title

19. abstract

20. keywords

21. introduction

22. related work

23. methodology

24. architecture/system design

25. experiments

26. results

27. discussion

28. limitations

29. conclusion

30. references

31. Detectar archivos bibliográficos:

32. .bib

33. references.bib

34. bibliography.bib

35. Guardar una versión intermedia en Markdown:

data/reviews/<review_id>/extracted/manuscript_extracted.md

1. Guardar metadatos de extracción en:

data/reviews/<review_id>/extracted/latex_extraction_report.md

Advertir si:

- no encuentra el archivo principal `.tex`
- hay múltiples candidatos principales
- faltan archivos incluidos
- faltan figuras referenciadas
- faltan archivos `.bib`
- hay comandos LaTeX personalizados que no puede interpretar
- la extracción puede haber perdido contenido

La compilación de LaTeX debe ser opcional y aislada.

Submission nueva vs resubmission

En Paso 1 el usuario debe poder indicar:

- new submission
- resubmission
- minor revision
- major revision
- reject and resubmit
- post-review revision
- camera-ready

Si es resubmission, permitir subir:

- reviews anteriores
- carta al editor
- respuesta a reviewers
- versión anterior del manuscript
- versión nueva del manuscript
- diff o lista de cambios

El sistema debe generar:

data/reviews/<review_id>/input/resubmission_context.md data/reviews/<review_id>/input/
previous_reviews_summary.md data/reviews/<review_id>/input/author_response_matrix.md data/
reviews/<review_id>/input/changes_claimed_by_authors.md

La matriz de respuesta a reviewers debe tener:

reviewer_id, comment_id, original_comment, author_response, claimed_change, manuscript_section, status, evidence_in_new_manuscript, remaining_risk

Los agentes revisores deben saber si están evaluando una submission nueva o una resubmission.

Si es resubmission, cada reviewer debe:

1. Revisar si el autor respondió a los puntos anteriores.
2. Verificar si los cambios están realmente en el manuscript.
3. Detectar inconsistencias nuevas.
4. Revaluar el paper completo.
5. Decidir si la respuesta a revisores es suficiente.

Pipeline web para el usuario

Implementa estos pasos.

Paso 0 — Crear revisión

- Crear ID único.
- Crear estructura documental.
- Registrar metadata.
- Inicializar estado.
- Mostrar dashboard inicial.

Paso 1 — Subida del paper

Permitir subir:

- PDF
- DOCX
- Markdown
- LaTeX
- ZIP con LaTeX
- ZIP con LaTeX + figuras + BibTeX

Permitir marcar si es:

- submission nueva
- resubmission
- major revision
- minor revision
- reject and resubmit
- camera-ready

Permitir subir reviews anteriores y carta/respuesta a reviewers.

Paso 2 — Extracción y clasificación automática

El sistema debe extraer:

- title
- abstract
- keywords
- research_area
- problem_statement
- claimed_contributions
- methodology
- architecture_or_system_design
- datasets
- experiments
- metrics
- baselines
- results_summary
- limitations
- reproducibility_artifacts
- ethical_considerations
- references
- paper_type
- likely_venue_families

Guardar:

data/reviews/<review_id>/extracted/paper_extraction.md data/reviews/<review_id>/extracted/
paper_extraction.json

Paso 3 — Screening editorial temprano / desk-reject precheck

Antes de ejecutar toda la revisión, hacer un precheck:

- ¿El paper está fuera de scope?
- ¿Falta contribución clara?
- ¿Falta evaluación experimental mínima?
- ¿El formato está muy lejos de la revista?
- ¿Hay referencias incompletas?
- ¿Hay claims sin evidencia?
- ¿La revista exige datos/código y el paper no los declara?
- ¿Hay problemas éticos evidentes?
- ¿El paper parece no estar listo para revisión?

Generar:

data/reviews/<review_id>/venues/desk_reject_precheck.md

Este paso no debe detener automáticamente el pipeline, pero debe advertir.

Paso 4 — Selección de revistas/congresos

El sistema debe mostrar:

- áreas detectadas
- venues candidatos
- Q1/Q2/accesible Q1
- tiempo estimado de publicación
- riesgo de desk reject
- encaje científico
- encaje por formato
- estrategia recomendada

El usuario puede:

- elegir una revista
- elegir varias revistas para comparación
- añadir una revista nueva
- abrir modal de configuración de revista
- importar revista desde CSV
- generar prompt externo para completar datos de revista

El proceso de evaluación debe poder avanzar revista por revista de forma secuencial.

Paso 5 — Selección de perfiles de reviewers

Por defecto todos marcados.

Reviewers principales:

1. reviewer-methodology
2. reviewer-domain
3. reviewer-systems-architecture
4. reviewer-reproducibility
5. reviewer-scientific-impact

Reviewers especializados activables:

- reviewer-document-ai-htr
- reviewer-education-research
- reviewer-dataset-benchmark
- reviewer-spectral-agri-food
- reviewer-fas-biometrics-security
- reviewer-cloud-hpc-scheduling
- format-compliance-checker
- integrity-ai-use-auditor

El usuario puede activar/desactivar reviewers.

Paso 6 — Generación de prompts externos por revista, motor y reviewer

El sistema debe generar prompts para:

- ChatGPT
- Claude
- Gemini
- Perplexity
- NotebookLM

Los prompts deben generarse por:

- review_id
- venue_id
- reviewer_profile
- engine

Estructura:

```
data/reviews/<review_id>/external_prompts/by_venue/<venue_id>/ |——  
rev1_methodology_chatgpt_prompt.md |—— rev1_methodology_claude_prompt.md |——  
rev1_methodology_gemini_prompt.md |—— rev1_methodology_perplexity_prompt.md |——  
rev1_methodology_notebooklm_prompt.md |—— rev2_domain_chatgpt_prompt.md |—— ...
```

Cada prompt debe especificar:

1. Nombre exacto del fichero esperado.
2. Rol del motor externo.
3. Datos del paper.
4. Datos de la revista.
5. Perfil del reviewer.
6. Criterios de evaluación.
7. Formato de salida en Markdown.
8. Prohibición de inventar referencias.
9. Petición de separar evidencia, inferencia y limitaciones.
10. Petición de indicar fuentes utilizadas.

Ejemplo de nombre esperado:

```
<review_id><venue_id><reviewer_profile>_<engine>_response.md
```

Ejemplo:

```
REV-20260607-143012-A8F3K2_ieee_toe_rev1_methodology_chatgpt_response.md
```

En la UI, mostrar una lista colapsable por revista:

Revista: <venue_name> Reviewer 1 — Methodology ChatGPT: copiar prompt | subir respuesta Claude: copiar prompt | subir respuesta Gemini: copiar prompt | subir respuesta Perplexity: copiar prompt | subir respuesta NotebookLM: copiar prompt | subir respuesta Reviewer 2 — Domain/SOTA ...

Plantilla profesional para prompts externos

Cada prompt externo debe seguir esta estructura:

You are acting as an independent scientific reviewer for a simulated pre-submission editorial review.

This is not a real peer-review process. Your goal is to help the author improve the manuscript before submission.

Review ID: <review_id>

Target venue: <venue_name>

Expected output filename: <expected_response_filename>

You must return your answer in Markdown. If your interface allows file generation, generate a Markdown file with the exact filename above. If not, print the full Markdown content so it can be copied and saved manually.

You are reviewing the manuscript according to the following venue information:

<venue_profile> <aims_and_scope> <author_guidelines> <review_criteria>
<publication_timeline_if_available>

Manuscript information:

<paper_extraction> <manuscript_summary> <claimed_contributions> <methods> <experiments>
<results> <limitations>

Reviewer profile:

<reviewer_profile_description>

Your task:

1. Evaluate whether the paper fits the target venue.
2. Evaluate the paper according to your reviewer profile.
3. Identify major strengths.
4. Identify major weaknesses.
5. Identify minor weaknesses.
6. Identify missing evidence.
7. Identify unsupported or exaggerated claims.
8. Identify questions for the authors.
9. Suggest concrete revisions.
10. Provide scores by dimension.

11. Provide a final recommendation:
12. accept
13. minor revision
14. major revision
15. reject
16. desk reject risk
17. Indicate your confidence.
18. Separate:
19. evidence from the manuscript
20. your inference
21. missing information
22. external knowledge, if used
23. Do not invent references, venue policies, statistics, quartiles or publication times.
24. If you use external information, provide source links.
25. If you cannot verify something, write NOT_VERIFIED.

Markdown output structure:

External Review

Metadata

- review_id:
- venue_id:
- venue_name:
- reviewer_profile:
- engine:
- expected_response_filename:
- sources_used:
- confidence:

Short summary

Venue fit

Major strengths

Major weaknesses

Minor weaknesses

Methodological concerns

Novelty and state of the art

Reproducibility concerns

Ethical or integrity concerns

Questions for authors

Required revisions

Optional improvements

Scores

Dimension	Score 1-10	Rationale
-----------	------------	-----------

Recommendation

Confidence and limitations

Evidence table

Claim / issue	Evidence from manuscript	Reviewer inference	Missing information
---------------	--------------------------	--------------------	---------------------

Final notes

Paso 7 — Importación de respuestas externas

Cuando el usuario sube una respuesta externa, el sistema debe:

1. Convertir PDF/DOCX a Markdown si es necesario.
2. Validar que corresponde al ID de revisión.
3. Detectar motor usado.
4. Detectar reviewer.
5. Detectar revista.
6. Crear resumen.
7. Extraer claims importantes.
8. Marcar si incluye fuentes.
9. Marcar si parece incompleta.
10. Guardar en:

`data/reviews/<review_id>/external_responses/by_venue/<venue_id>/`

Generar:

`data/reviews/<review_id>/external_responses/index.md`

Paso 8 — Pendencias / requests de agentes

Los agentes pueden pedir información adicional al usuario.

Cada agente puede pedir como máximo 3 iteraciones por defecto.

Debe ser configurable en `pipeline.yaml`.

Las pendencias deben aparecer en una tabla arriba de la página de revisión.

Campos:

- pending_id
- review_id
- venue_id
- agent_id
- reviewer_profile
- request_type
- question
- suggested_engine
- generated_prompt_path
- status
- iteration_number
- max_iterations
- upload_button

Tipos de pendencia:

- missing venue information
- missing publication timeline
- missing SOTA
- missing author guidelines
- missing format template
- missing recent papers
- missing dataset information
- missing resubmission context
- missing review response
- clarification required

El usuario puede subir un fichero como respuesta a cada pendencia.

Paso 9 — Ejecución autónoma de subagentes

Los subagentes se ejecutan de forma independiente.

Reglas:

- Un reviewer no debe leer la salida de otro reviewer.
- Puede leer el paper.
- Puede leer datos de la revista.
- Puede leer literatura permitida.
- Puede leer respuestas externas asignadas a su rol.
- Puede pedir información adicional hasta el máximo configurado.
- Debe indicar modelo/motor/fuentes/limitaciones.
- Debe diferenciar submission nueva de resubmission.

Los reviewers principales son:

Reviewer 1 — Methodology and experimental validity

Evalúa:

- validez interna
- diseño experimental
- métricas
- datasets
- baselines
- ablation studies
- análisis estadístico
- claims no soportados
- amenazas a la validez

Reviewer 2 — Domain expert and state of the art

Evalúa:

- novedad
- estado del arte
- papers recientes
- contribución científica
- si el trabajo es incremental
- relevancia para la comunidad
- adecuación al venue

Si la matriz de literatura está vacía, debe decir que la evaluación de novedad es provisional.

Reviewer 3 — Systems, architecture and implementation

Evalúa:

- arquitectura del sistema
- pipeline
- escalabilidad
- complejidad computacional
- deployment
- cloud/edge/hardware implications
- latencia
- memoria
- energía
- throughput
- factibilidad de ingeniería

Reviewer 4 — Reproducibility and empirical robustness

Evalúa:

- reproducibilidad
- disponibilidad de código/datos
- hiperparámetros
- seeds
- splits train/val/test
- protocolo experimental
- fairness de benchmarks
- robustez
- sensitivity analysis
- replicabilidad

Reviewer 5 — Scientific impact and editorial strategy

Evalúa:

- si el paper merece existir

- si la contribución es suficientemente fuerte
- si el paper tendría impacto
- si sería citado
- si el storytelling científico es convincente
- si el paper encaja con la audiencia del venue
- si el envío es estratégico
- si conviene cambiar de venue

Reviewers especializados

reviewer-document-ai-htr

Activar cuando el paper trate sobre HTR, OCR, document analysis, synthetic handwriting, handwriting datasets, low-resource languages, document AI o child handwriting.

Evalúa:

- calidad del dataset
- protocolo de anotación
- split writer-independent
- métricas CER/WER
- comparación con IAM, RIMES, CVL, Bentham, READ, ICFHR/ICDAR benchmarks u otros datasets relevantes
- validez de datos sintéticos
- brecha de dominio entre sintético y real
- protocolos de fine-tuning
- reproducibilidad del pipeline
- licencias de datos, fuentes textuales y tipografías
- privacidad si hay datos de menores
- adecuación a venues de document analysis

reviewer-education-research

Activar cuando el paper trate sobre computing education, educational technology, K-12, STEM, programming education, computational thinking, curriculum analysis o engineering education.

Evalúa:

- claridad de research questions
- marco teórico educativo
- diseño metodológico
- PRISMA si es systematic review
- validez de instrumentos
- nivel educativo: primary, secondary, VET, higher education
- intervención educativa
- resultados de aprendizaje
- amenazas a la validez en educación
- ética, consentimiento y menores
- adecuación a educational technology / CS education / engineering education venues

reviewer-dataset-benchmark

Activar cuando el paper presente un dataset, benchmark, corpus o data descriptor.

Evalúa:

- novedad del recurso
- documentación del dataset
- licencias
- FAIRness
- reproducibilidad
- splits
- leakage-free protocols
- baselines
- metadata
- provenance
- benchmark difficulty
- community reuse
- data availability statement
- target venue tipo Scientific Data, Data in Brief, SoftwareX, Journal of Open Source Software, GigaScience o Patterns

reviewer-spectral-agri-food

Activar cuando el paper trate sobre NIR-HSI, hyperspectral imaging, DON, Fusarium, wheat, food safety, mycotoxins, chemometrics, calibration transfer o precision agriculture.

Evalúa:

- validez del diseño experimental
- calibración
- adquisición espectral
- batch/session effects
- leakage risks
- grouped evaluation
- chemometric baselines
- food safety relevance
- agricultural deployment
- relación con Food Control, Food Chemistry, Computers and Electronics in Agriculture, Remote Sensing, Sensors, Chemometrics and Intelligent Laboratory Systems, Journal of Food Engineering o Journal of Cereal Science

reviewer-fas-biometrics-security

Activar cuando el paper trate sobre FAS, PAD, biometrics, face recognition security, spoofing, deepfake, video identity verification o lightweight biometric systems.

Evalúa:

- protocolos FAS/PAD

- datasets FAS
- ataques conocidos/desconocidos
- generalización cross-dataset
- métricas APCER/BPCER/ACER/AUC/EER
- real-time constraints
- coste computacional
- robustez frente a ataques
- relevancia biométrica y de seguridad

reviewer-cloud-hpc-scheduling

Activar cuando el paper trate sobre cloud scheduling, CloudSim, distributed systems, HPC, orchestration, energy-aware computing o scheduling policies.

Evalúa:

- diseño del scheduler
- workloads
- trazas reales vs sintéticas
- fairness
- starvation
- latencia
- throughput
- energía
- reproducibilidad de simulación
- seeds
- intervalos de confianza
- comparación con baselines fuertes

Paso 10 — Auditoría de integridad, IA, ética y coherencia

Crea un auditor separado:

integrity-ai-use-auditor

No debe actuar como reviewer normal. Debe actuar como checker editorial.

Debe evaluar:

- referencias sospechosas o incompletas
- referencias posiblemente inventadas
- claims sin evidencia
- inconsistencias entre abstract, método, resultados y conclusión
- contribuciones exageradas
- texto excesivamente genérico
- posible uso de IA no declarado
- transparencia sobre uso de IA
- problemas éticos

- data leakage
- plagiarism risk indicators
- dual-use concerns
- limitaciones ausentes
- detalles de reproducibilidad ausentes
- cumplimiento de author guidelines
- coherencia del título
- coherencia del abstract
- limpieza textual
- tono académico
- claridad de figuras/tablas
- acknowledgements y authorship issues
- privacidad y consentimiento si hay datos de menores
- licencias de datasets, fuentes textuales, imágenes, tipografías y código

Guardar:

data/reviews/<review_id>/reviewer_outputs/integrity_ai_use_audit.md

Integra también la skill:

.claude/skills/review-paper-reviewer/SKILL.md

Paso 11 — Editor-in-chief

El editor debe leer:

- paper_extraction
- venue_profile
- venue_fit_report
- venue_timeline_report
- cinco reviews independientes
- reviews especializados
- respuestas externas importadas
- integrity audit
- literature matrix
- resubmission context, si existe
- pending requests resueltas

Debe producir:

data/reviews/<review_id>/editor/editor_decision.md data/reviews/<review_id>/editor/meta_review.md
data/reviews/<review_id>/editor/revision_plan.md data/reviews/<review_id>/editor/
rebuttal_strategy.md data/reviews/<review_id>/editor/final_letter.md

La decisión debe ser una de:

- desk reject
- reject
- major revision

- minor revision
- accept

El editor debe:

- explicar desacuerdos entre revisores
- identificar debilidades decisivas
- identificar fortalezas reales
- decidir si el venue es adecuado
- recomendar redirección a otro venue si procede
- distinguir revisiones obligatorias de mejoras opcionales
- crear un plan de revisión ordenado por prioridad
- crear una estrategia de respuesta a revisores
- indicar qué debe cambiar para que el paper sea acceptable

No hacer promedio mecánico.

Paso 12 — Export final

Exportar:

```
data/reviews/<review_id>/exports/ ├── final_editor_decision.md ├── final_editor_decision.pdf
├── revision_plan.md ├── reviewer_response_matrix.md ├── rebuttal_letter_draft.md ├──
venue_fit_report.md ├── audit_log.md └── full_review_package.zip
```

Comparación con Llama/Ollama

Crea:

```
scripts/compare_models.py
```

Debe permitir:

```
provider: claude | ollama model: configurable input_prompt: path output_file: path
```

Guardar comparaciones en:

```
data/reviews/<review_id>/reviewer_outputs/model_comparison/
```

El informe debe comparar:

- severidad
- especificidad
- riesgo de alucinación
- utilidad
- problemas omitidos
- consistencia con la rúbrica
- acuerdo/desacuerdo con Claude

Docker Compose

Crea `docker-compose.yml` con servicios:

- web
- api
- worker
- postgres
- ollama opcional
- storage local mediante volumen

Usa volúmenes persistentes para:

- postgres_data
- review_data
- upload_data
- ollama_data

GitHub privado

Prepara `.gitignore` para no subir por defecto:

- data/reviews/
- data/uploads/
- PDFs privados
- ZIPs privados
- .env
- claves
- modelos locales
- archivos temporales

Permite versionar:

- código
- templates
- prompts
- perfiles genéricos de reviewers
- esquemas
- configuración de ejemplo
- documentación

Si se quieren versionar archivos grandes, documentar uso de Git LFS, pero no activarlo automáticamente.

Scripts mínimos

Implementa estos scripts aunque inicialmente algunas partes solo creen archivos plantilla:

- scripts/init_review.py

- scripts/run_pipeline.py
- scripts/parse_paper.py
- scripts/discover_venues.py
- scripts/scan_venue_markdowns.py
- scripts/build_literature_matrix.py
- scripts/compare_models.py
- scripts/build_report.py
- scripts/export_review_package.py

El script `parse_paper.py` debe incluir:

- detect_input_format()
- validate_upload()
- extract_from_markdown()
- extract_from_pdf()
- extract_from_docx()
- extract_from_latex()
- extract_from_latex_zip()
- detect_main_tex_file()
- resolve_latex_inputs()
- extract_bib_files()
- convert_latex_to_intermediate_markdown()

El comando principal CLI debe ser:

```
python scripts/run_pipeline.py --review-id <review_id> --mode full_review
```

Modos soportados:

- init
- ingest
- validate_upload
- extract
- scan_venues
- discover_venues
- venue_fit
- timeline
- literature_queries
- generate_external_prompts
- import_external_responses
- review
- specialized_review
- integrity
- editorial_decision
- export
- full_review

README

Crea un README.md explicando:

1. Objetivo del sistema.
2. Arquitectura.
3. Cómo ejecutar con Docker Compose.
4. Cómo crear una revisión.
5. Cómo subir un paper Markdown.
6. Cómo subir un paper PDF.
7. Cómo subir un paper DOCX.
8. Cómo subir un paper LaTeX.
9. Cómo subir un ZIP con LaTeX.
10. Cómo indicar que es resubmission.
11. Cómo subir reviews previas.
12. Cómo subir carta a reviewers.
13. Cómo configurar revistas.
14. Cómo importar revistas desde CSV.
15. Cómo añadir papers recientes / SOTA.
16. Cómo generar prompts externos.
17. Cómo subir respuestas externas.
18. Cómo gestionar pendencias.
19. Cómo ejecutar reviewers internos.
20. Cómo ejecutar reviewers especializados.
21. Cómo ejecutar auditoría de integridad.
22. Cómo generar decisión editorial.
23. Cómo exportar paquete final.
24. Cómo comparar Claude con Ollama.
25. Qué archivos se pueden subir a GitHub.
26. Qué archivos no deben subirse a GitHub.

Primera tarea

Construye la primera versión funcional del proyecto.

Prioriza un MVP robusto:

1. Estructura de carpetas.
2. Docker Compose.
3. Backend mínimo FastAPI.
4. Frontend mínimo con layout lateral y wizard de revisión.
5. Creación de review ID.
6. Subida segura de archivos.
7. Conversión/extracción básica a Markdown.
8. Configuración de venues.
9. Generación de prompts externos.
10. Importación de respuestas externas.
11. Pipeline interno basado en Markdown.
12. Subagentes/reviewers como archivos `.md`.

13. Auditoría final.
14. Export final.
15. README.

Después de crear los archivos, ejecuta una verificación básica de estructura y muestra:

- archivos creados
- cómo ejecutar con Docker Compose
- cómo crear una revisión
- cómo subir un paper
- cómo añadir revistas
- cómo generar prompts externos
- cómo subir respuestas externas
- cómo ejecutar el pipeline interno
- cómo ver historial
- próximos pasos recomendados